M16 Multi-Agent: Topology & Orchestration

Methodology Advanced

Architecting agent teams for complex tasks

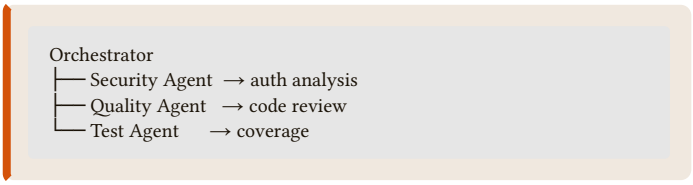
When to switch to multi-agent

Switch to multi-agent work when the work decomposes into independent scopes, outputs can be verified separately, and coordination adds more than it costs. Repository size, context usage, and agent count do not provide universal thresholds.

Signal	Single agent	Team
Coherent, sequential scope	Preferred	Unnecessary overhead
Independent, verifiable research	Possible	May reduce elapsed time
Shared files or state	Preferred	Conflict risk
Complex recovery and synthesis	Possible	An explicit orchestrator may help

Star topology

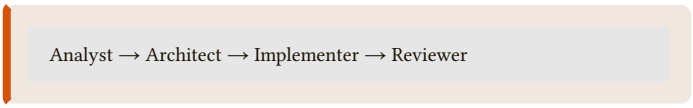
The central orchestrator receives the global objective, breaks it down into subtasks, delegates to specialized agents, then synthesizes their results. This is the default topology for Claude Code Agent Teams.



In this chosen arrangement, the orchestrator focuses on planning and delegation; this is a role convention, not a tool restriction. Its role is to allocate work, unblock dependencies, and present a unified response.

Pipeline topology

Agents work in sequence: the output of one becomes the input of the next. Useful when steps are strictly ordered and each result conditions the next.



Advantage: each agent focuses on a precise phase, without noise from previous phases in its context. Disadvantage: a blockage on one step stops the entire pipeline.

Parallel topology

Independent agents work simultaneously on modules with no dependencies between them. Typical case: refactoring frontend, backend and infra in parallel on non-shared files.



Applicability condition: modules must have zero shared state. If agents constantly need to read each other's outputs or modify common files, git conflicts and coordination messages cancel out the gain.

Isolation and permissions per agent

Each agent can receive distinct context and a tool whitelist. Limiting an agent to `Read`, `Glob`, `Grep` tools prevents accidental modification during the analysis phase.

Agent	Allowed tools
Explorer	Read, Glob, Grep
Planner	Read, Glob, Grep; separately scoped plan-file writes
Implementer	Read, Edit, Bash, Write
Reviewer	Read, Glob, Grep

Enabling agent teams

```
# Environment variable (current session)
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
claude

# Persistent configuration ~/.claude/settings.json
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Prerequisites: Claude Code v2.1.32+, a model available to the account (teammates may differ); a Git repository is required for worktree isolation, not for task messaging itself. Navigate between agents with `Shift+Down` in in-process mode.

Loop controls

Define a budget, stop condition, recovery state, and synthesis owner before launch. Add an independent reviewer when the change is consequential or when the same agent cannot credibly produce and verify the result.

Limit new work to verification capacity; start with the smallest team covering independent scopes. Add a participant only when elapsed time, conflicts, human rework, and accepted-task cost justify the extra coordination. The trial protocol provides the record for that decision.